Автономная некоммерческая организация высшего образования
«Университет Иннополис»
(АНО ВО «Университет Иннополис»)

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(БАКАЛАВРСКАЯ РАБОТА)
по направлению подготовки
09.03.01 – «Информатика и вычислительная техника»

GRADUATION THESIS
(BACHELOR'S GRADUATE THESIS)
Field of Study
09.03.01 – «Computer Science»

Направленность (профиль) образовательной программы
«Информатика и вычислительная техника»
Area of Specialization / Academic Program Title:
«Computer Science»

Тема / Topic	**Локальные LLM vs general-модели vs SAST в гибриде: снижение FP и артефакты безопасности в embedded C/C++ (IoT) / Local LLMs vs general models vs SAST, hybrid: reducing FPs and persistent security artifacts in embedded C/C++ (IoT)**

Работу выполнил / Thesis is executed by	**Нгуен Илья-Линь Куанг Хынгович / Nguyen Ilya-Linh Quanh Hungovich**	подпись / signature
Руководитель выпускной квалификационной работы / Graduation Thesis Supervisor	**Андрей Садовых / Andrey Sadovykh**	подпись / signature
Консультанты / Consultants		подпись / signature

Иннополис, Innopolis, 2026

Contents

1 Introduction **3**
1.1 Background . . . 3
1.2 Problem Statement . . . 3
1.3 Research Questions . . . 4
1.4 Objectives . . . 4
1.5 Approach Overview . . . 5
1.6 Contributions . . . 5
1.7 Scope and Limitations . . . 6

2 Literature Review **7**
2.1 Static Analysis and Its Noise . . . 7
2.2 Learned Detectors and Language Models . . . 8
2.3 Hybrids of Static Analysis and Language Models . . . 9
2.4 Locally Deployable Vulnerability Models . . . 9
2.5 From Findings to Executable Artifacts . . . 10
2.6 Where a Persistent Check Would Have to Live . . . 11
2.7 Benchmark Quality and Training-Data Leakage . . . 11
2.8 Position of This Thesis . . . 12

Introduction

1.1 Background

Embedded devices run code written in C and C++, a language family without built-in bounds checking. A missing length check in a packet parser is enough to make a device remotely exploitable, and such defects are common: a systematization of 15,116 publicly reported IoT vulnerabilities covering the years 2013 to 2024 identifies memory-safety weaknesses as the largest single class of IoT flaws [37]. The affected code usually lives in network stacks, protocol parsers and cryptographic libraries of real-time operating systems such as Zephyr, RIOT-OS and Contiki-NG, or of TLS implementations such as wolfSSL and Mbed TLS.

Automated auditing of this code relies on static application security testing (SAST). Tools such as CodeQL [21], Semgrep [38], cppcheck [10] and SonarQube [40] scale well and are cheap per line of code, yet their practical value is limited by noise rather than by blindness. Developers asked for the suppression of false alarms long before they asked for more findings [8], and empirical classification of analyzer issues confirms that a large share of reported warnings is dismissed [11]. Container and dependency scanners inherit a related structural limitation: they match package metadata against a CVE database, so software compiled from source and installed outside a package manager stays invisible to them. A study of 50 Docker images reports up to 415 additional vulnerabilities beyond those found by Trivy on a single image [25]. Embedded firmware is a harder version of the same problem, because it normally has no package manifest at all.

Language models trained on vulnerability data offer a way past pattern matching, but the practical ones are cloud services, and sending firmware source to a third-party API conflicts with vendor confidentiality and export restrictions. This constraint has produced a new family of compact models that run locally: CodeAstra-500M [19], R2Vul-1.5B [46] and VulnLLM-R-7B [36] detect or reason about vulnerabilities in C/C++ on laptop hardware, and knowledge-tuned models such as Foundation-Sec-8B [9] write security documentation locally. Detection quality of such models remains far from solved: on a statement-level benchmark of 5,867 real-world C/C++ CVEs, the strongest model evaluated reaches an F1 score of 23.83 [1].

1.2 Problem Statement

The difficulty this thesis addresses is not primarily about finding more vulnerabilities. It is about what remains in a repository after a finding is produced.

A static analyzer reports a location and a rule identifier. It misses the context that decides whether the location matters in a firmware build: conditional compilation through `#ifdef` directives, protocol implementations that exist only in one project, and defects that depend on hardware peripherals such as I2C or UART. The consequence is the noise described above, and a noisy tool is eventually switched off by the team that was supposed to benefit from it.

A language model does not fix this with a single run either. Sampling makes its output unstable, so the same function can be judged vulnerable in one run and benign in the next [41]. Its answer is a report in a console: once the process ends, nothing is left in the repository, and the next commit

starts the analysis from zero. Specialized local models are no exception here; they differ in what they say, not in how long the statement survives.

The gap is therefore about persistence rather than about detection. A finding becomes useful to a project when it is converted into an executable check that runs on every future commit without the model that produced it. Producing such checks automatically is not new: agents synthesize CodeQL queries from CVE reports [45], generate regression tests from commit diffs [33], reproduce CVEs as runnable proofs of concept [20, 43], and distill Semgrep rules from patch [28]. What is missing is a study of this conversion for embedded C/C++ code, performed entirely by locally deployed models, and a measurement of the quality of what survives the conversion.

Two further problems make the question non-trivial. First, models trained on vulnerability corpora may recognize a known CVE instead of reasoning about the code in front of them, which inflates every reported quality figure; the training sets of the leading specialized models include the same datasets that later studies evaluate on [6, 12, 36]. Second, existing comparisons measure how well a model detects, so they cannot answer which configuration of models produces better artifacts. This thesis changes the unit of measurement from the finding to the artifact.

1.3 Research Questions

The main question of the thesis is whether a cascade of locally deployed language models can turn a security finding into a durable, model-independent check in embedded C/C++ projects, and which configuration of models does it best. It is decomposed into three sub-questions.

RQ1. *How do vulnerability-specialized local models, general-purpose local models, a static analyzer, and their hybrid compare in finding real CVEs in embedded C/C++ code?* The answer is measured by recall and precision, and by the false-positive count produced at equal recall.

RQ2. *Which combination of local models (specialized detectors, a general coder, an explainer) produces valid artifacts (rules, tests, harnesses) that work without an LLM on future commits?* The answer is measured by the Artifact Validity Rate (AVR), the share of generated artifacts that pass execution-based validation.

RQ3. *Does the resulting capability transfer to CVEs published later than the training cutoff of the model, or does the model reproduce an answer it memorized?* The answer is measured by comparing AVR on recent CVEs with AVR on older ones.

RQ1 establishes which model deserves which role in the pipeline. RQ2 is the central question of the thesis, because the contribution claimed here is the artifact set rather than the detection score. RQ3 protects the interpretation of RQ2: a high AVR is meaningful only if it does not depend on memorization.

1.4 Objectives

To answer these questions, the thesis pursues the following objectives.

1. Build a corpus of vulnerable and patched commit pairs from open embedded C/C++ projects, restricted to memory-safety weakness classes, and split it per model against the training cutoff declared for that model.

2. Implement a pipeline that filters candidates deterministically, audits them with a specialized local model over repeated runs, and distills each confirmed finding into five kinds of artifacts: a Semgrep rule, a CodeQL query, a regression test compiled with a sanitizer, a fuzzing harness, and a hardware mock, together with advisory documentation.

3. Define a validation criterion for each artifact that requires no language model, based on running the artifact against the vulnerable commit, the patched commit, and a set of clean files.
4. Compare four configurations on the same corpus: specialized models alone, general-purpose models alone, the analyzer alone, and the hybrid pipeline.
5. Measure AVR overall, per artifact kind, and on the recent slice of the corpus, and publish the corpus, the artifacts and the evaluation scripts.

1.5 Approach Overview

The design follows one principle: a model proposes, execution decides. The pipeline runs in two loops with different cost profiles. The commit loop executes only the artifacts that already exist and needs no model, so its cost is a normal build. The distillation loop runs on new candidates and uses models in four stages. A deterministic rule filter over dangerous API calls reduces the candidate set. A 500M-parameter specialized model filters the remainder over five runs and aggregates the verdicts. A 7B reasoning model audits the survivors over thirty runs, with duplicates merged by file, function and CWE class. Each confirmed finding is then handed to the model suited to its output type: a code model writes rules, harnesses and mocks, the reasoning model writes tests, and a security-tuned model writes advisory text.

Every generated artifact is validated by execution, without a model. A rule must match the vulnerable version, must not match the patched version, and must stay silent on clean files. A test must fail under a sanitizer on the vulnerable commit and pass on the patched one. A harness must reproduce a crash within a bounded run, and a mock must compile with the same signature as the peripheral it replaces. Artifacts that fail are discarded and counted; artifacts that pass are committed to the repository under a `security/` directory and run in continuous integration from that moment on.

The evaluation corpus consists of approximately 170 CVEs from eight open embedded projects, each represented by a vulnerable and a patched commit. Models are used as published, without fine-tuning, and run on a laptop with 16 GB of unified memory, with 7B and 8B models quantized to four bits.

1.6 Contributions

The expected contributions of the thesis are the following.

1. **A metric for the persistence of a finding.** AVR measures the share of generated artifacts that work without a model, which shifts the evaluation question from which model detects best to which configuration leaves a usable trace in a project.
2. **A pipeline that distills findings of local models into five kinds of executable security artifacts** for embedded C/C++, with role-based routing between specialized detectors, a general-purpose code model and a security knowledge model.
3. **A contamination-controlled corpus** of approximately 170 vulnerable and patched commit pairs from eight IoT projects, together with a per-model date split and an exclusion list of datasets that overlap model training data.
4. **An empirical comparison** of specialized models, general-purpose models, a static analyzer and the hybrid pipeline, reported with execution-based validation and with the memorization check of RQ3.

5. **A public artifact set** of validated rules, tests and harnesses, released with the evaluation scripts so that the numbers can be reproduced.

No new model is trained, and the contribution is not a detector. The claim is that a fixed set of published local models, arranged in a cascade and gated by execution, produces checks that outlive the run that created them.

1.7 Scope and Limitations

The study is limited to C and C++ source code of open embedded projects, with memory-safety weakness classes as the target set. Binary firmware analysis, hardware-in-the-loop testing and network fuzzing of a running device are out of scope. Models are used zero-shot or with prompt engineering; no fine-tuning, distillation or retrieval augmentation is evaluated, so the results say nothing about whether a smaller model could be trained to do the same job.

Artifact validation is execution-based, which makes it objective but not complete: a test that passes on a pair of commits may still be weak, and advisory text requires human review, so documentation artifacts are scored separately from executable ones. The corpus is bounded by what the selected projects disclose, which limits the number of recent CVEs available for the memorization check. Findings reported here describe the models and the corpus evaluated and are not generalized to other languages or domains.

Literature Review

Chapter 1 located the problem in what happens to a security finding after it is produced, not in how the finding is produced. Framed that way, the question touches six threads of literature: the noise behavior of static analyzers, detection by learned models and language models, hybrids of the two, locally deployed vulnerability-specialized models, the generation of executable artifacts from findings, and the quality of the datasets on which all of the above is measured.

The review covers publications from 2016 to 2026 and was assembled in three passes. Two recent maps served as the starting point: a systematic literature review of 263 studies on vulnerability detection with language models [24], and a survey that organizes the same field by technique and lists open problems [39]. From the studies these two identify as central, I followed the citation chains forward and backward, kept the works that report an execution-based result, and added domain evidence for embedded code and for the delivery pipelines where a check would have to live. Each thread below ends with what it contributes to the design in Chapter 3 and with what it leaves unanswered.

2.1 Static Analysis and Its Noise

Rule-based and query-based analyzers are the mature half of this field. CodeQL [21] compiles a repository into a queryable database and expresses detections as queries over it, which makes dataflow and taint analysis declarative and reusable. Semgrep [38] matches patterns and simple taint rules over source trees, is fast, and covers many languages with a single engine. cppcheck [10] and the C/C++ ruleset of SonarQube [40] trade precision for coverage of common defect patterns such as unchecked buffers and misuse of dangerous library calls. Comparative work on Java tools distinguishes these two families cleanly: rule matching finds local patterns cheaply, while taint tracking follows values across calls at a higher analysis cost and a higher false-positive rate [29].

The cost of that design is documented, and it is not primarily blindness but noise. A multi-method study at Microsoft invited 2,000 developers to a survey, obtained 375 responses, added interviews and an analysis of corrected defects, and reports that developers are willing to trade analysis time for results with fewer false positives and false negatives, and that they value deterministic output from an analyzer [8]. An analysis of issue trackers from open projects reaches a compatible conclusion from the other side: the reasons analysts and developers give for dismissing or missing reports follow a pattern rooted in the inability of a tool to model context rather than in the absence of a rule [11]. A tool that produces noise is configured down and then switched off, so the finding rate of a checker says little about its value to the team operating it.

A second, structurally different blind spot matters for the domain of this thesis. Container and dependency scanners identify software components from package metadata and compare the result against a CVE database. Tools of this kind, among them Trivy [4] and Grype [3], therefore cannot report anything about software that was compiled from source and installed outside the package manager. Khasanov and Sadovykh measure the consequence on 50 Docker images: after parsing the image build history, isolating layers that install software manually and scanning the extracted files by version strings, a single image yielded up to 415 vulnerabilities beyond those

reported by Trivy, with medians of 29 additional findings on popular images and 38.5 on randomly selected ones [25]. The method they build on is DAVS, which derives a list of potentially vulnerable files from Dockerfile instructions and scans it separately from the package manifest [13]; the same authors evaluate DAVS on real images and catalog its limits, namely the need for a readable Dockerfile, multi-stage builds and binaries that carry no version string [26].

Embedded firmware is a harder instance of the same problem. A firmware image usually has no package manifest at all: third-party code is vendored into the source tree, patched by hand, and recompiled, so metadata matching has nothing to work with, while the interesting defects sit in the vendored C files. The conclusion I take from this thread is that a scanner reading metadata cannot protect a device, and that source-level analysis is required; what this thread does not answer is what to do with the finding once it exists.

2.2 Learned Detectors and Language Models

Detection by learned models predates language models. Devign represents a function by its control-flow and data-flow graphs and classifies it with a graph neural network, reporting strong accuracy on curated data [49]. Chakraborty et al. examined the same generation of results and argued that the field was measuring the wrong thing: labels were noisy, random splits let near-duplicate functions leak between training and test sets, and accuracy on synthetic data did not transfer to real projects [6]. That criticism defines the methodological contract that every later benchmark, including the corpus used in this thesis, is judged against.

Language models changed the shape of the task, because a model can be prompted instead of trained. The two maps cited above agree on the taxonomy of approaches: prompting with contextualization, fine-tuning on vulnerability corpora, and multi-agent or tool-assisted pipelines, with contamination between training data and evaluation data and repository-level reasoning named as open problems [24, 39]. Decoding strategy is a variable of its own: a controlled comparison of chain-of-thought, tree-of-thoughts and self-consistency on the Juliet suite shows that aggregating repeated derivations makes verdicts more stable at the price of additional computation [41]. This is the direct precedent for the repeated sampling and aggregation used in my pipeline.

When a language model and an analyzer are run on identical inputs, the model usually finds more and reports it more noisily. On 63 vulnerabilities taken from ten C# projects, the F1 score of language models reaches 0.75 to 0.80, against 0.26 to 0.55 for SonarQube, CodeQL and Snyk Code, while the models produce more spurious findings and localize defects worse [22]. A study of 32 scenarios comparing GPT-4 with SonarQube applies the McNemar test to the paired outcomes, which is the statistical treatment I adopt for comparing the analyzer configuration with the hybrid one [42]. On real C code, a benchmark of 250 programs evaluated 13 static analyzers and ten language models with a composite score, again with complementary strengths rather than a winner [15].

Absolute detection quality on real C/C++ remains low. SecVulEval collects 25,440 function samples covering 5,867 CVEs from 1999 to 2024 and asks a model to point at the vulnerable statement rather than to label the function; the best configuration evaluated, a strong cloud model, reaches an F1 score of 23.83 [1]. Two readings of this number are relevant here. First, the question of which model detects best is open, so RQ1 is not a solved comparison. Second, even the strongest detector in that setting outputs a judgment about a function, which is exactly the artifact that does not survive its own run.

2.3 Hybrids of Static Analysis and Language Models

The third thread combines the two preceding ones: an analyzer produces candidates and a language model supplies the context the analyzer lacks. IRIS asks GPT-4 to deduce the taint specifications CodeQL needs and to filter the resulting alerts; on CWE-Bench-Java it recovers 55 of 120 known vulnerabilities where CodeQL alone recovers 27, with a false-discovery rate of roughly 13% against 27% for the analyzer [31]. Sift compares agentic frameworks for triaging CodeQL alerts: on the OWASP Benchmark the false-positive share of retained alerts falls from above 92% to 6.3%, and on OSS-Fuzz C/C++ projects the agents detect 95.5% of false positives while keeping precision at 95.5% [47]. AdaTaint addresses the hallucination risk directly with a neuro-symbolic division of labor: the model proposes sources and sinks, and the taint engine validates the proposed paths under constraint checks before any report is emitted [32].

These three studies justify two design decisions in my pipeline. The pattern deterministic-filter-then-model-audit is empirically sound, and the model must be gated by a mechanism it cannot talk its way past, which in my case is compilation and execution rather than a constraint solver. The limitation is shared and rarely stated: the output of a hybrid is a verdict on an alert. The Sift agent decides that an alert is noise, IRIS decides that a path is feasible, and in both cases the reasoning is discarded when the run finishes. The next commit triggers the same analysis over the same code. None of the three writes anything back into the repository, and all three rely on cloud models, which is the constraint that pushes this thesis toward local ones.

2.4 Locally Deployable Vulnerability Models

Confidentiality is the reason local models matter for embedded vendors: sending firmware source to a third-party API is normally not acceptable, and export restrictions and air-gapped build environments make the cloud option moot. A family of compact models now runs on laptop hardware.

CodeAstra publishes a 500M-parameter detector and a 7B variant. The smaller model starts from a Mixture-of-Experts base, is trained on a 140,335-row vulnerability corpus strongly skewed toward C, and returns a structured verdict naming the vulnerability, the CWE class and the OWASP category; the larger one applies LoRA fine-tuning to Mistral-7B across 13 languages and reports accuracy near 83% on CVEfixes-derived data [19]. R2Vul trains a 1.5B-parameter model with reinforcement learning from AI feedback and distillation of structured reasoning traces, and reports that the student outperforms its 32B teacher, using 18,000 multilingual preference pairs and a threshold-calibration step that trades verdict volume for precision without losing recall [46]. VulnLLM-R-7B is fine-tuned for vulnerability reasoning on PrimeVul, ARVO, Juliet 1.3 and SVEN, emits an explicit reasoning chain, and was wrapped by its authors in an agent scaffold that identified unpatched defects in maintained repositories [36]. Antares targets repository-scale agentic localization rather than function classification: its 350M, 1B and 3B models build on Granite 4.0 and are trained with verifiable rewards, but on application code in Python and JavaScript, which places them off-domain for embedded C [44].

A separate branch is tuned for security knowledge rather than for classification. Foundation-Sec-8B continues pre-training of Llama-3.1-8B on approximately 5.1 billion tokens of cybersecurity text, including CVE descriptions, advisories and standards, and is strong at advisory and documentation tasks [9]. CyberAssist-mini is a lighter model with the same orientation, knowledge and explanation rather than detection [2]. Neither is a candidate for the detector role; both are candidates for the writing role in a role-based cascade, which is the distinction my pipeline relies on. Qwen2.5-Coder-7B-Instruct serves as the general-purpose control: a code model with no security-specific training [23].

Two observations follow. The first is that detector and writer are different models, so a single-model comparison answers a different question than the one RQ2 asks. The second is that every

model in this thread publishes a per-invocation judgment. Their output is text about a function, produced once, and the specialized ones inherit from their training corpora the contamination problem discussed in Section 2.7. None of them was proposed as a generator of persistent checks.

2.5 From Findings to Executable Artifacts

This thread is the closest to the contribution claimed here, and it is the youngest.

QLCoder synthesizes CodeQL queries from CVE metadata inside an agentic loop with execution feedback, using a language-server interface for syntax and retrieval over existing queries for semantics. Evaluated on 176 CVEs across 111 Java projects, it produces a query that flags the vulnerable version and stays silent on the patched version for 53.4% of them, against 10% for an unassisted coding agent [45]. Two things make this the reference point for RQ2: the definition of a correct query is a definition I reuse verbatim for rules, tests and harnesses, and the gap between 53.4% and 10% quantifies how much of the value of artifact generation comes from the validation loop rather than from the model. Its limits mark the three axes on which this thesis differs: the target language is Java, the model is a cloud service, and rules are the only output.

Autogrep distills Semgrep rules from vulnerability patches at scale, generating candidates from 39,931 patches and filtering them by the same firing criterion, which yields 645 rules across 20 languages [28]. The input differs from mine in an important way: a patch has to exist already, so the technique produces checks for defects that are fixed upstream, not for what an audit of the current tree finds.

Cleverest generates regression tests from commit diffs, evaluated on 72 commits across eight programs, most of them C libraries and interpreters such as MuJS, libxml2, JerryScript and MicroPython; test generation takes under two minutes where a prior-art-based approach needs about 24 hours, and coupling generation with fuzzing doubles the number of bugs found [33]. The objective function differs from mine: a commit-diff test protects behavior that already worked, whereas my test must fail on the vulnerable commit and pass on the patched one, which is a security oracle rather than a behavioral one.

CVE-Genie reproduces a proof of concept from the text of a CVE report through a four-stage agent whose last stage verifies the result by execution against a flag, recovering 428 of 841 CVEs, about 51%, spanning 267 projects, 141 CWE classes and 22 languages; without the scaffolding, a strong reasoning model fails on the same input [43]. PoC-Gym generates proofs of concept for Java from static source-to-sink traces and filters them at runtime: of 338 generated attempts, 116 run validly, 65 survive post-hoc checks, and 12 become new CVEs [20]. The funnel is the sharpest available evidence for the premise of this thesis: a generation pipeline without execution validation is mostly noise, and a large fraction of what looks like a finding is not executable at all. Earlier work on supply-chain attacks shows the same pattern with humans in the loop: pairing ChatGPT with security testers produced 24 demonstrable proofs of vulnerability against CI/CD infrastructure, four of them assigned CVEs [48].

LLIoT is the corresponding domain evidence for the Internet of Things. It systematizes how IoT vulnerabilities are studied, analyzes 15,116 publicly reported IoT CVEs from 2013 to 2024, builds a layered taxonomy of IoT-specific weaknesses and classifies them with a language model at about 95% accuracy, concluding that memory-safety problems are the largest single class and that the field lacks dependable IoT datasets [37]. That last finding is why the corpus of this thesis is assembled from commit pairs rather than drawn from an existing benchmark. The work uses a language model to describe CVEs, not to turn a finding into a check, and it is the gap between those two statements that RQ2 addresses.

2.6 Where a Persistent Check Would Have to Live

An artifact that survives the run still has to be executed somewhere. The delivery pipeline is the only place that sees every change, so the question of whether CI is a trustworthy enforcement point is in scope. An analysis of 447,000 GitHub Actions workflow files identifies injection and lateral-movement risks that make the pipeline itself an attack surface [27]; Argus applies taint analysis specialized to workflow scripts, which is a precedent for building a checker for the pipeline language rather than reusing a general one [34]; GHAST statically analyzes workflows and catalogs security-relevant misconfigurations [5]; and a comparison of nine scanners for workflow files shows how widely the same repository can be judged depending on the tool chosen [17].

For this thesis the point is narrower. The commit-side gate must be cheap, deterministic and independent of a model, because it runs on every push; the expensive part belongs to a separate, less frequent loop. The CI literature describes how to secure that gate, and none of it addresses what to put inside it for embedded C/C++, which is what the distillation loop is for.

2.7 Benchmark Quality and Training-Data Leakage

Every measurement reported above depends on data, and the data in this field has known defects. Big-Vul links C/C++ fix commits to CVE entries and became the standard change-level corpus [16]; DiverseVul deduplicates CVE sources into 23,668 CVE records with the intent of covering more projects [7]; Juliet supplies synthetic but CWE-complete programs for C, C++ and Java [35]. Chakraborty et al. showed why such corpora overstate performance: near-duplicates across a random split and noisy labels turn the test set into a second training set [6].

PrimeVul treats the problem as a dataset-engineering task, building vulnerable and patched function pairs, enforcing chronological splits so that a model is always tested on the future, and auditing label correctness, reporting roughly 92% correctness where earlier corpora sit far lower; the accompanying result is that published accuracies drop substantially once the split is honest, and that the best model on this benchmark still leaves the task open [12]. CleanVul pushes label quality further with LLM-based heuristics over commit messages and diffs, reaching 90.6% correctness on 8,198 function-level pairs, against 60 to 80% for the corpora it cleans [30]. SastBench applies the same logic to triage benchmarks, selecting CVEs published after the training cutoff of the evaluated agents so that a memorized answer is not available [18].

The consequence for model comparison is direct, because the specialized local models of Section 2.4 are trained on exactly these corpora. VulnLLM-R uses PrimeVul, ARVO, Juliet 1.3 and SVEN [36]; CodeAstra uses a corpus whose functions come from the Linux kernel, Chromium, PHP and ffmpeg [19]; R2Vul contributes its own 18,000-pair multilingual dataset [46]; Foundation-Sec-8B continues training of a Llama-3.1 base whose knowledge ends in December 2023 and states a cutoff of 10 April 2025 [9, 14], while Qwen2.5-Coder reports a cutoff of February 2024 [23]. Any of these models can therefore recognize a known CVE instead of reading the code in front of it, and a benchmark drawn from its own training data cannot expose that.

I take three precautions from this thread. The corpus is built as chronological pairs in the style of PrimeVul and CleanVul. Evaluation reports results on CVEs published later than the cutoff declared for the model under test, per model rather than per benchmark. Datasets that appear in the training data of a model are excluded from the slice used for that model. This is also the reason RQ3 exists as a separate question: without it, a high AVR would be unreadable as evidence of capability.

2.8 Position of This Thesis

Table 2.1 places the closest works against the five properties that matter for the question posed in Chapter 1: what a work consumes, what it leaves in the repository, whether the result is checked by execution, whether the models run locally, and whether embedded C/C++ is in scope.

Table 2.1: Closest works: input, persistent output, execution validation, model locality and target domain

Work	Consumes	Leaves behind	Checked by execution	Models local	Embedded C/C++
IRIS [31]	CodeQL alerts, CWE descriptions	triaged alert list	no	no	no, Java
Sift [47]	CodeQL alerts	triage verdict per alert	no	no	partial, OSS-Fuzz C/C++
AdaTaint [32]	candidate sources and sinks	validated taint path	engine check	no	no
QLCoder [45]	CVE report	CodeQL query	yes	no	no, Java
Autogrep [28]	existing patch	Semgrep rule	yes	no	no, 20 languages
Cleverest [33]	commit diff	regression test	yes, compile and run	no	partial, C libraries
CVE-Genie [43]	CVE report	proof of concept	yes	no	no, 22 languages
PoC-Gym [20]	static source-to-sink trace	proof of concept	yes	no	no, Java
VulnLLM-R [36], CodeAstra [19]	function source	verdict text	no	yes	yes
LLIoT [37]	CVE database	taxonomy labels	no	not applicable	yes
This thesis	finding of a local model	rule, query, test, harness, mock, advisory	yes, no model	yes	yes

Read by column rather than by row, the table shows three regularities. Execution validation is now standard wherever an executable object is produced, and absent wherever the output is a judgment. Locality and validation sit on opposite sides: the works that validate by execution use cloud models, whereas the works that run locally produce no validated artifact. Target-domain evidence for embedded C/C++ exists for detection [1, 37] and for generation [15, 33], but no study connects the two.

The gap this thesis addresses is therefore specific and can be stated in one sentence: no prior work converts a finding produced by a fully local, vulnerability-specialized model into a set of executable security artifacts for embedded C/C++ code, and no metric exists to compare such conversions. Existing evaluation practice supports the first half of that sentence and undermines the second: detection is measured by F1 over functions [1, 6, 12], hybrids are measured by false-positive reduction [31, 47], and generators are measured by the share of outputs that succeed on their own target [20, 43, 45]. What is missing is a measure of how much of what a model produces remains usable after the model is switched off, which is the Artifact Validity Rate defined in Chap-

ter 3 and used to answer RQ2. The three research questions follow from the table in order: RQ1 compares the detectors that the table lists as local, RQ2 asks which configuration of them produces the row labeled *This thesis*, and RQ3 tests whether the contamination discussed in Section 2.7 explains the result.

References

[1] M. B. U. Ahmed et al. SecVulEval: Benchmarking LLMs for real-world C/C++ vulnerability detection, 2025. URL https://arxiv.org/abs/2505.19828.

[2] AI Security Assistant. CyberAssist-mini: A lightweight language model for vulnerability knowledge and explanation. https://github.com/Ai-Security-assistant/CyberAssist, 2025. Accessed: 2026-10.

[3] Anchore. Grype: Version vulnerability scanner for images and filesystems. https://github.com/anchore/grype, 2025. Accessed: 2026-10.

[4] Aqua Security. Trivy: Container and filesystem vulnerability scanner. https://github.com/aquasecurity/trivy, 2025. Accessed: 2026-10.

[5] Gabriele Benedetti, Giuseppe Verderame, and Alessio Merlo. Automatic security assessment of GitHub actions workflows (GHAST), 2022. URL https://arxiv.org/abs/2208.03837.

[6] Preetha Chakraborty et al. Deep learning based vulnerability detection: Are we there yet? *IEEE Transactions on Software Engineering*, 2021. URL https://arxiv.org/abs/2009.07235.

[7] Yizheng Chen et al. DiverseVul: A new vulnerable source code dataset for deep learning based vulnerability detection. In *Research in Attacks, Intrusions, and Defences (RAID)*, 2023. URL https://arxiv.org/abs/2304.00409.

[8] Maria Christakis and Christian Bird. What developers want and need from program analysis: An empirical study. In *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)*. ACM, 2016. doi: 10.1145/2970276.2970347.

[9] Cisco Foundation AI. Foundation-Sec-8B: An open-source large language model for cybersecurity. https://huggingface.co/fdtn-ai/Foundation-Sec-8B, 2025. Model card. Accessed: 2026-10.

[10] Cppcheck team. Cppcheck: Static analysis of C/C++ code. https://cppcheck.sourceforge.io, 2025. Accessed: 2026-10.

[11] Haopeng Cui et al. Empirical study of false negatives and false positives of static code analyzers, 2024. URL https://arxiv.org/abs/2408.13855.

[12] Yifeng Ding et al. Vulnerability detection with code language models: How far are we? (PrimeVul). In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, 2025. URL https://arxiv.org/abs/2403.18624.

[13] Thanh Phuong Doan and Sebastian Jung. DAVS: Dockerfile analysis for container vulnerability scanning. *Computers, Materials & Continua*, 72(1):1699–1711, 2022.

[14] Abhimanyu Dubey et al. The Llama 3.1 herd of models, 2024. URL https://arxiv.org/abs/2407.21783.

[15] Daniel Dubniczky et al. CASTLE: Code analysis and security testing language evaluation, 2025. URL https://arxiv.org/abs/2503.09433.

[16] Jinghui Fan, Yu Li, Shaohua Wang, and Tuan Anh Nguyen. A C/C++ Code Vulnerability Dataset with Code Changes and CVE Summaries (Big-Vul). In *Proceedings of the 17th International Conference on Mining Software Repositories (MSR)*, 2020. URL https://huggingface.co/datasets/bstee615/bigvul.

[17] Mahmoud Fares, Yapesa Gamage, and Benoit Baudry. Unpacking security scanners for github actions workflows, 2026. URL https://arxiv.org/abs/2601.14455.

[18] Jonathan Feiglin and Gat Dar. SastBench: A benchmark for testing agentic SAST triage, 2026. URL https://arxiv.org/abs/2601.02941.

[19] Hari Sundar Ganesan. CodeAstra-500M and CodeAstra-7B: Laptop-scale vulnerability detection models. https://huggingface.co/rootxhacker/codeastra-500M, 2026. Model card. Accessed: 2026-10.

[20] Deniz Gezgin et al. PoC-Gym: Towards more reliable LLM-assisted proof-of-concept exploit generation, 2026. URL https://arxiv.org/abs/2602.04165.

[21] GitHub. CodeQL language and security documentation. https://codeql.github.com/docs, 2025. Accessed: 2026-10.

[22] Dariusz Gnieciak and Tomasz Szandala. Large language models versus static code analysis tools: A systematic benchmark for vulnerability detection. *IEEE Access*, 13:198410–198422, 2025. doi: 10.1109/ACCESS.2025.3635168.

[23] Binyuan Hui et al. Qwen2.5-Coder technical report, 2024. URL https://arxiv.org/abs/2409.12186.

[24] Sebastian Kaniewski et al. A systematic literature review on detecting software vulnerabilities with large language models. *ACM Transactions on Software Engineering and Methodology*, 2025. doi: 10.1145/3815425. 263 studies. Preprint: https://arxiv.org/abs/2507.22659.

[25] Ilnar Khasanov and Andrey Sadovykh. Improving vulnerability analysis of docker images. *ACM Ada Letters*, 2025. doi: 10.1145/3821489.3821505. Dataset available at https://zenodo.org/doi/10.5281/zenodo.15301809.

[26] Ilnar Khasanov and Andrey Sadovykh. Evaluating DAVS approach for docker images static analysis. In *Computer Analysis of Images and Patterns (ICCSA)*, Lecture Notes in Computer Science. Springer, 2026. doi: 10.1007/978-3-032-00644-8_11.

[27] Beket Koishybayev, Michael Fazzini, Alexander Apalkov, Andrew Ormrod, Paul Van Keuren, Alessandro Paccione, Jonathan M. Smith, and David Evans. Attack surface analysis of github actions. In *31st USENIX Security Symposium*, 2022. URL https://www.usenix.org/conference/usenixsecurity22/presentation/koishybayev.

[28] Lambda Security. Autogrep: Automated generation and filtering of semgrep rules from vulnerability patches. https://github.com/lambdasec/autogrep, 2025. Accessed: 2026-10.

[29] Jinghui Li, Shangxin Chen, et al. Comparison and evaluation on static application security testing (SAST) tools for Java. In *Proceedings of the ACM on Software Engineering (FSE)*, 2023.

[30] Yuandao Li et al. CleanVul: Automatic function-level vulnerability detection in code commits using LLM heuristics, 2024. URL https://arxiv.org/abs/2411.17274.

[31] Zian Li, Saikat Dutta, and Mayur Naik. IRIS: LLM-assisted static analysis for detecting security vulnerabilities, 2024. URL https://arxiv.org/abs/2405.17238.

[32] Shangnin Lin et al. AdaTaint: LLM-driven adaptive source-sink identification and false positive mitigation, 2025. URL https://arxiv.org/abs/2511.04023.

[33] Jiwei Liu, Seungjin Lee, Elena Losiouk, and Marcel Böhme. Evaluating large language model-based regression test generation (Cleverest). *Proceedings of the ACM on Software Engineering*, 2026. doi: 10.1145/3808129.

[34] Mathias Muraleedharan et al. Argus: Detecting vulnerabilities in CI/CD workflow templates. In *32nd USENIX Security Symposium*, 2023. URL https://www.usenix.org/conference/usenixsecurity23/presentation/muralee.

[35] National Institute of Standards and Technology. SAMATE Juliet 1.3.0 test suite for C/C++. https://samate.nist.gov/SARD/test-suites, 2017. Accessed: 2026-10.

[36] Yang Nie et al. VulnLLM-R: Specialized reasoning large language model with agent scaffold for vulnerability detection, 2025. URL https://arxiv.org/abs/2512.07533.

[37] Mohammad Rezaei et al. Sok: Understanding the state of IoT-specific vulnerabilities. 2026. URL https://conand.me/publications/rezaei-lliot-2026.pdf.

[38] Semgrep. Semgrep documentation: Static analysis with custom rules. https://semgrep.dev/docs, 2025. Accessed: 2026-10.

[39] Zeyu Sheng et al. Large language models in software security: A survey of vulnerability detection techniques, 2025. URL https://arxiv.org/abs/2502.07049.

[40] SonarSource. SonarQube Static Analysis for C/C++. https://sonarqube.org, 2025. Accessed: 2026-10.

[41] Karl Tamberg and Harun Bahsi. Harnessing large language models for vulnerability detection: Chain-of-thought, tree-of-thoughts and self-consistency. *IEEE Access*, 13:29698–29717, 2025. doi: 10.1109/ACCESS.2025.3541146.

[42] Mohit S. Tehrani et al. LLM vs. SAST: GPT-4 versus SonarQube in paired scenarios, 2025. URL https://arxiv.org/abs/2506.15212.

[43] Saheed Ullah et al. From CVE entries to verifiable exploits: A multi-agent framework for reproducing CVEs, 2025. URL https://arxiv.org/abs/2509.01835.

[44] Srivatsa Vijay et al. Antares: Foundation models for agentic vulnerability localization, 2026. URL https://arxiv.org/abs/2608.02407.

[45] Xiang Wang, Zian Li, Saikat Dutta, and Mayur Naik. QLCoder: A query synthesizer for static analysis of security vulnerabilities, 2026. URL https://arxiv.org/abs/2511.08462.

[46] Martin Weyssow et al. R2Vul: Learning to reason about software vulnerabilities with reinforcement learning and structured reasoning distillation, 2025. URL https://arxiv.org/abs/2504.04699.

[47] Ying Xiong and Tianyin Zhang. Sift: A comparative study of LLM agents in vulnerability false positive filtering. In *Proceedings of the 35th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2026. URL https://arxiv.org/abs/2601.22952.

[48] Yu Zhang et al. How can chatgpt support human security testers to mitigate supply chain attacks?, 2023. URL https://arxiv.org/abs/2310.00710.

[49] Yi Zhou et al. Effective vulnerability identification by learning system-level code representation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. URL https://arxiv.org/abs/1909.03496.